

Separation of Concerns for Identifiers in USD

Copyright © 2026, NVIDIA Corporation, version 0.1 (DRAFT)

Aaron Luk

Contents

- [Introduction](#)
- [Motivation](#)
 - [USD identifiers today](#)
 - [The expanding ecosystem](#)
- [Problem statement](#)
 - [Two distinct roles for identifiers](#)
 - [Why this matters now](#)
- [Key questions](#)
 - [Instance identity vs. source identity](#)
 - [Single value vs. metadata package](#)
- [Existing mechanisms in USD](#)
 - [assetInfo, UsdModelAPI, and UsdMediaAssetPreviewsAPI](#)
 - [displayName](#)
 - [customData and customLayerData](#)
- [Industry use cases](#)
 - [Architecture, Engineering, Construction, and Operations \(AECO\)](#)
 - [Manufacturing, Product Lifecycle, and Digital Engineering](#)
 - [Media and Entertainment \(M&E\)](#)
- [Design considerations](#)
 - [Principles](#)
 - [Open questions for discussion](#)
 - [Likely direction](#)
 - [Risks](#)
- [Relationship to other proposals](#)
- [Next steps](#)
- [Appendix A: AI-Assisted Drafting](#)

Introduction

As OpenUSD adoption grows across industries, a recurring tension has emerged between the identifiers that USD uses internally for scene composition and hierarchy navigation, and the identifiers that external systems use to track assets, components, and objects. These two kinds of identifiers serve fundamentally different purposes, yet there is no clear, standardized mechanism in USD to represent and manage external identifiers alongside USD's namespace paths.

Discussions around this topic have revealed that two related but distinct problems are often conflated:

1. **An unencumbered source identifier field** -- a standardized place to store external identifiers verbatim, outside the prim name, free from USD grammar constraints.
2. **Improved ergonomics of path identifiers** -- extending USD's prim name grammar to natively support constructs like leading digits and medial hyphens, improving the usability of namespace paths themselves.

Both are valuable. This proposal focuses on problem (1) because an unencumbered source identifier field addresses the broadest set of cross-industry use cases and can be pursued independently of grammar changes. Problem (2) -- extending prim name grammar -- remains an important complementary effort and is not foreclosed by anything proposed here. By establishing consensus on the separation of concerns first, both problems can be pursued on their own merits without one blocking or distorting the other.

Expected outcome. The likely result is a standardized mechanism for source identifiers in the OpenUSD codebase, accompanied by a baseline example that domains can follow. Two candidate approaches are under evaluation -- extending `assetInfo` with stratified sub-dictionaries, or leveraging USD's existing applied schema mechanism with typed properties -- each with distinct trade-offs (see [Likely direction](#)). This proposal seeks consensus on the problem statement and design principles first, so that the resulting mechanism serves the full community.

Motivation

USD identifiers today

In USD, every prim is addressed by a **namespace path** -- a hierarchical, textual identifier such as `/world/Building/Floor1/Room101`. These paths are the backbone of USD's composition engine: references, payloads, inherits, variants, and overrides all bind to prims by their namespace paths.

USD's namespace paths are:

- **Unique per instance** within a composed stage.
- **Structural** -- they encode the scene hierarchy and are used for traversal,

queries, and composition arc targeting.

- **Governed by grammar rules** -- identifiers must conform to syntactic constraints (currently the XID specification as of OpenUSD v24.03).
- **The primary key** for binding opinions across layers.

A recurring community question is

[why USD does not use GUIDs](#)

as primary identifiers. While GUIDs would make rename and refactor workflows more ergonomic, they cannot replace namespace paths: composition depends on paths as **sort keys** for deterministic ordering of opinions across layers, and opaque GUIDs would destroy that ordering. USD addresses the rename problem through

[relocates](#)

instead.

The GUID debate is itself an example of the conflation this proposal seeks to resolve. Much of the pressure for GUIDs-as-primary-identifiers comes not from a desire to replace namespace paths, but from the absence of any standardized place to carry stable external identifiers *alongside* them. A dedicated source identifier field would address that underlying need without disturbing the composition model.

The expanding ecosystem

USD was born in the visual effects and animation industry, where scene hierarchies tend to originate from digital content creation (DCC) tools with naming conventions that align well with USD's identifier grammar. As USD expands into new industries, it increasingly encounters data originating from systems with their own identification schemes:

- **AECO tools** (Revit, Archicad, IFC) use identifiers like GUIDs, room numbers (1001), classification codes with slashes and hyphens (BB/500), and revision-stamped names.
- **Manufacturing and Product Lifecycle Management (PLM) systems** (Teamcenter, Windchill, 3DEXPERIENCE) track parts by alphanumeric part numbers (A-0000-12345), revision identifiers, component designators with leading digits and medial hyphens (1N4148 , R-101), and multi-attribute metadata packages. The same assets increasingly participate in digital engineering loops, where equipment tags and sensor addresses bind USD scene objects to live telemetry streams.
- **GIS and infrastructure tools** use identifiers tied to geospatial coordinate systems or regulatory codes.
- **Game engines and M&E pipelines** use asset management systems with their own unique IDs, tags, and versioning metadata.

In all these cases, the external identifier is *not* an alternative name for the same prim -- it identifies something conceptually different: the **source asset, component, or entity** in the originating system, which may be instantiated multiple times in a USD stage, or may carry metadata that has no

equivalent in the USD namespace.

Problem statement

Two distinct roles for identifiers

The core observation is that there are two fundamentally different roles an identifier can play in a USD-based workflow, and that conflating them has led to a single conversation trying to solve two distinct problems at once: the need for an unencumbered source identifier field, and the desire for improved prim name ergonomics. Separating these concerns allows each to be addressed on its own terms.

	USD namespace identifier	Source / external identifier
Purpose	Address a prim in the composed stage	Identify an asset, component, or entity in an external system
Uniqueness	Unique per prim instance in the stage	• Unique per source entity, not per prim instance. • May be shared across instances (e.g., multiple placements of the same part). • Must remain stable through state transitions, re-parenting, and assembly changes.
Governed by	USD prim name grammar (XID rules)	External system conventions (may include characters invalid in USD namespace identifiers)
Used for	Composition, hierarchy traversal, overrides	Asset tracking, classification, BOM generation, regulatory compliance, cross-system linking
Persistence	Tied to the prim's position in the namespace	Tied to the source entity; should survive namespace edits
Multiplicity	Exactly one per composed prim	Potentially many -- one per external system the entity participates in

Today, there is no standardized place in USD to express the right-hand column. Users work around this through ad-hoc conventions: encoding external identifiers in prim names (losing characters to grammar restrictions), storing them in `customData` (losing discoverability and interoperability), or using `displayName` (conflating display concerns with identification).

Why this matters now

Without a clear separation of concerns:

1. **Data loss on ingest.** External identifiers that contain characters invalid in USD prim names (leading digits, hyphens, slashes, GUIDs) are either altered or lost when data enters USD. Round-tripping back to the source system becomes unreliable.
2. **Ad-hoc solutions fragment the ecosystem.** Each integrator invents its own

convention for storing external identifiers, making cross-tool interoperability difficult. A Revit-to-USD pipeline and a Teamcenter-to-USD pipeline may store their source identifiers in incompatible ways.

3. **Conflation of concerns.** When prim names are forced to double as external identifiers, the USD namespace becomes polluted with naming constraints from external systems, or external systems lose fidelity when their identifiers are forced into USD grammar. There is also a fundamental **uniqueness incompatibility**: if multiple instances of the same part share an external identifier (e.g., identical bolts in an assembly), they must either have uniquified USD paths to coexist as siblings, or be placed under artificial namespace structure -- regardless of how rich USD's character set becomes.
4. **Missed opportunities for tooling.** If source identifiers were discoverable through a standard mechanism, tools could provide cross-system linking, BOM generation, classification lookups, and regulatory compliance checks without requiring knowledge of each pipeline's ad-hoc conventions.
5. **The `displayName` migration is already underway.** The [UI Hints](#) work ([implemented in OpenUSD](#)) has migrated `displayName` into a `uiHints` dictionary, cementing it as a presentation-only concern. The old top-level accessors are on a deprecation path. Pipelines using `displayName` to carry source identifiers now face **deprecation-driven migration costs** -- and without a standardized alternative, they will migrate to yet another ad-hoc convention that will need to be migrated again later.

Key questions

Before proposing a specific solution, the community should align on two foundational questions.

Instance identity vs. source identity

Namespace paths already serve as instance identifiers -- they uniquely address a specific prim in a specific composition. The gap is in *source* identity: which external asset, component, or entity does this instance represent?

Consider a building where the same door type is placed in 30 locations. Each placement has a unique namespace path, but they all share a source identity: the door type's catalog number or PLM part number. That source identity is independent of the namespace -- it should survive renames and be shared across instances.

Some domains also assign external instance-level identifiers -- serial numbers for individual physical parts, or IFC GlobalIds that uniquely identify each placement rather than the shared type. A flexible mechanism should accommodate both source-level and instance-level external

identifiers, but the primary gap today is source identity. (Note that instance-level external identifiers raise additional questions about behavior under reparenting and namespace edits -- constraints that would likely need to be enforced by DCCs and validators rather than by core namespace editing logic.)

Single value vs. metadata package

Some external identification schemes are simple strings -- a GUID, a part number, a classification code. Others are richer: a dictionary of metadata that together constitutes the identity in the external system, potentially including identifiers from multiple systems simultaneously.

For example, a single structural column in a building might carry:

System	Identifier type	Value
IFC	GlobalId	202Fr\$t4X7Zf8NOew3FNr2
Revit	ElementId	847562
Uniclass 2015	Classification	Ss_25_10_30
Revit	Mark	C-14

<!--

- For Uniclass 2015 codes, see: [Uniclass 2015 - Table Ss: Structure](#) for example codes.
- The "Project"/"Mark" example was originally included to demonstrate a project-specific identifier. In AECO (Architecture, Engineering, Construction, and Operations) workflows, "Mark" is a commonly used parameter (e.g., in Autodesk Revit) for an instance or tag number. "Project" is not a standardized system like IFC or Revit, so "Revit / Mark" better reflects actual usage, where the "Mark" parameter is an identifier for individual elements.

-->

The need for multiple identifiers is not limited to design-time systems. In digital engineering workflows, the same physical asset may also carry operational bindings -- an equipment tag from a building management system, a sensor address from a telemetry platform, or a work-order ID from a maintenance system (see [Composable Bindings](#), Microsoft and NVIDIA, 2025). A single prim hierarchy representing a chiller unit might carry its PLM part number, its IFC GlobalId, *and* its OPC UA NodeId -- each serving a different system integration.

This points toward a mechanism that can hold multiple identifiers from different systems, keyed by domain -- whether as a composed dictionary (extending `assetInfo`) or as typed properties on an applied schema (see [Likely direction](#)).

Existing mechanisms in USD

Several existing mechanisms partially address the need for external identifiers. Understanding their strengths and limitations informs the evaluation of candidate approaches (see [Likely direction](#)).

assetInfo, UsdModelAPI, and UsdMediaAssetPreviewsAPI

`assetInfo`

is a composed dictionary that can technically live on any prim, though its convenience API is provided through

`UsdModelAPI`,

which is designed for prims representing the root of a *model* (as defined by USD's `Kind` hierarchy). `UsdModelAPI` exposes a small set of well-known

`assetInfo` keys:

- `identifier` (`SdfAssetPath`) -- the asset's resolvable path.
- `name` (`string`) -- the asset's name, suitable for database queries.
- `version` (`string`) -- the asset's resolved version.
- `payloadAssetDependencies` (`SdfAssetPath[]`) -- dependencies inside the payload.

`assetInfo` is composed element-wise and is nestable, which makes it well-suited for model-level asset management metadata.

`UsdMediaAssetPreviewsAPI`

demonstrates an extension of this pattern: it is an applied API schema that provides typed access to a nested sub-dictionary of `assetInfo` (`assetInfo["previews"]["thumbnails"]`). However, it is important to note that such applied schemas contribute nothing to a prim's *definition* (`UsdPrimDefinition`). They provide a described spec and a convenience API, but not defining data types that can be reasoned about independently of specific C++/Python API calls -- no fallback values, no automatic GUI presentation of unauthored properties, no schema-driven validation.

Together, `UsdModelAPI` and `UsdMediaAssetPreviewsAPI` inform the design space but each has limitations:

- `UsdModelAPI` is designed around USD's own concept of an asset (a referenceable layer or package), not around external system identifiers. While `assetInfo` itself is not restricted to model roots, the `UsdModelAPI` convenience layer encourages use only on prims that participate in the `Kind` hierarchy. (Note: `assetInfo` is registered in `SdfSchema` for both prims and properties, suggesting it was intended to be broadly applicable. The API's current limitation to model roots appears to be an API gap rather than a design constraint, and could potentially be addressed by migrating the `assetInfo` API from `UsdModelAPI` to `UsdObject`.)

- Many real-world objects that need source identifiers (individual rooms, structural members, electrical components) are not model roots.
- The existing `assetInfo` keys are oriented toward Pixar's asset management model. Supporting arbitrary external identifiers from multiple systems would require either new sub-dictionary conventions or applied schemas with typed properties directed at this problem.

These patterns inform two candidate approaches for source identifiers -- extending `assetInfo` with stratified sub-dictionaries, or leveraging applied schemas with typed properties -- each with distinct trade-offs (see [Likely direction](#)).

displayName

`displayName` is metadata available on prim specs and property specs. It provides a human-readable name that can contain any UTF-8 string, including characters not valid in prim names.

`displayName` is useful for presentation but is semantically a *display* concern, not an *identification* concern. Using it to carry source identifiers conflates two purposes: the name shown to a user in a UI may differ from the identifier used to link back to a source system. A structural column's display name might be "Column C-14 (Level 3)" while its source identifier is the IFC GlobalId `202Fr$t4X7Zf8N0ew3Fnr2`.

The [UI Hints](#) work, now [implemented in OpenUSD](#),

has migrated `displayName` into a `uiHints` dictionary alongside other presentation-only metadata like `hidden` and `displayGroup`. This makes explicit what was always implicit: `displayName` is a UI concern. The new access API is

[UsdUIObjectHints](#),

and the old top-level accessors are deprecated. Pipelines that have been using `displayName` as a carrier for source identifiers now face an active migration -- and the question is whether they migrate to yet another ad-hoc convention, or to a standardized source identifier mechanism.

customData and customLayerData

`customData` is a freeform dictionary available on any prim or property.

`customLayerData` is the layer-level equivalent. Both are intended for pipeline-specific or ad-hoc metadata.

While `customData` can technically store external identifiers, it provides no standardization: every pipeline must agree on key names, and tools have no way to discover or interpret source identifiers without prior knowledge of the convention used. This is precisely the interoperability gap that a standardized mechanism would close.

Industry use cases

The following examples illustrate the problem across industries. They are not exhaustive but are intended to show that the need for separating USD namespace identifiers from source identifiers is broad and cross-cutting.

Architecture, Engineering, Construction, and Operations (AECO)

AECO data originates from tools like Autodesk Revit, Graphisoft Archicad, Bentley MicroStation, and the open IFC standard. Naming conventions in AECO frequently conflict with USD's prim name grammar:

- **Room numbers** are often purely numeric (e.g., `1001`), which cannot serve as a prim name starting character under pre-v24.03 rules and still cannot serve as identifiers under current XID rules.
- **Classification codes** use slashes, hyphens, spaces, and other delimiters not valid in USD prim names (e.g., `CI/SfB BB/500`, `OmniClass 23-13 11 00`).
- **IFC GlobalIds** are 22-character base64-encoded strings that uniquely identify building elements across the lifecycle of a project.
- **Revision workflows** produce new identifiers with each design iteration; the ability to derive revision information from the identifier is a requirement, not a convenience.

The AOUSD AECO Interest Group has documented these requirements in detail. A key insight from that work is the distinction between the **prim name** (which may be a transcoded or generated valid identifier) and the **source name** from the originating system, which must be preserved exactly for round-trip fidelity.

Manufacturing, Product Lifecycle, and Digital Engineering

PLM systems assign stable identifiers -- part numbers, revision codes, serial numbers -- that must persist across the lifecycle of a physical product and its digital twin.

- A **part number** like `A-0000-12345-Rev.C` identifies the *design* of a component. Multiple instances of that component in an assembly share this identifier.
- A **serial number** like `SN-2025-00847` identifies a specific *physical instance* of that component.
- A **BOM (Bill of Materials)** is generated by traversing an assembly and collecting part numbers -- a workflow that requires source identifiers to be discoverable and unambiguous.

In complex configuration-managed systems, the identifier may not be a human-readable part number at all. PLM systems may use composite URIs or opaque system keys -- meaningful only to the originating system's asset resolver -- where a "part" is an abstract container whose concrete identity

depends on configuration rules and context. These opaque, vendor-specific identifiers must survive a round-trip through USD without loss, just as human-readable strings do.

If these identifiers are encoded into prim names, characters like hyphens and periods are lost or transcoded, making BOM generation from the USD stage unreliable without an additional decoding step.

Identity continuity

In aerospace and defense manufacturing, the stakes are especially high: a part moves through multiple suppliers, routings, and process steps -- including destructive geometric changes like machining, heat treatment, and stress relief -- while its identity must remain stable for safety certification, fleet risk management, and root cause analysis.

The Association for Manufacturing Technology (AMT) has identified this as **identity continuity**: source identifiers that survive state transitions, re-parenting, and assembly changes without relying on the prim's position in the namespace.

Feature-level identifiers (individual holes, datum faces, tolerances) add a further dimension, since features may appear, disappear, or deform across manufacturing operations while remaining traceable.

Operational telemetry and composable bindings

Beyond design-time identification, the same assets increasingly participate in live operational loops. USD scenes representing real-world equipment (compute racks, robotic work cells, HVAC units) must bind to external systems streaming telemetry -- sensor readings, performance metrics, event logs -- where each event carries an **object identifier** (rack ID, sensor address, equipment tag) that must resolve to the correct prim hierarchy.

The [Composable Bindings](#) whitepaper (Microsoft and NVIDIA, 2025) describes this pattern in the context of the digital engineering loop: standardized source identifiers on prims would replace today's brittle, ad-hoc integration code with declarative telemetry-to-scene mappings.

Media and Entertainment (M&E)

Even in USD's original domain, asset management systems assign identifiers that differ from prim namespace paths:

- **Asset management database IDs** track versions, approvals, and dependencies at a granularity that may not align with USD's namespace hierarchy.
- **Shot and sequence identifiers** follow studio-specific conventions that may include characters not valid in prim names.
- **Published asset versions** are tracked by systems (e.g., Flow Production Tracking, ftrack)

that assign their own unique identifiers alongside the USD asset path.

While `assetInfo` covers some of these cases at the model level, it does not address identification at finer granularities (individual props, lights, materials) or across multiple asset management systems.

Across all three domains, the common thread is **traceability**: the ability to follow an entity from its origin in an external system, through its representation in USD, and back again -- regardless of namespace edits, assembly changes, or geometric mutations along the way.

Design considerations

This section outlines principles and open questions to guide the community toward a solution. The goal of this proposal is to establish consensus on the problem statement and separation of concerns *before* committing to a specific mechanism -- not because a solution is distant, but because premature implementation without that consensus is what produced the current landscape of fragmented workarounds.

Principles

1. **Separation of concerns.** USD namespace paths and external source identifiers serve different purposes. They should be stored and accessed through distinct mechanisms, even if they sometimes carry the same value.
2. **Industry agnosticism.** The mechanism should not be specific to any one industry's identifier scheme. It should be flexible enough to carry IFC GUIDs, PLM part numbers, M&E asset database IDs, and schemes not yet envisioned.
3. **Vendor extensibility.**
 - Any vendor, standards body, or consortium -- from a single company to an industry standard like IFC -- should be able to declare its own identifier scheme without central approval before deployment.
 - Identifiers may be opaque -- meaningful only to the declaring system. Support for such proprietary schemes is necessary; centralizing their management is neither desirable nor required.
 - When a vendor scheme matures to the point of requiring systems interoperability, proven extensions can be promoted to multi-vendor or core status over time -- a model with broad precedent, e.g.:
 - Khronos glTF (vendor prefix → `EXT_` → `KHR_`)
 - Khronos graphics APIs (OpenGL, Vulkan)
 - IETF internet standards
 - The W3C web platform

Of these, glTF is the closest analogy: it is a 3D data interchange format (not a graphics API), and its extension model governs *properties on objects* -- analogous to attaching vendor-defined

identifier metadata to USD prims. glTF also distinguishes governed `extensions` from freeform `extras`, paralleling the tension between schema-based metadata and `customData` in USD. The details of a vendor extension model for source identifiers -- prefix conventions, registration process, promotion criteria -- are out of scope for this proposal and would be addressed in a follow-up.

4. **Composability.** External identifiers should participate in USD's composition model in a well-defined way. It should be clear how source identifiers are resolved when a prim is referenced, inherited, or specialized.
5. **Discoverability.** Tools should be able to discover that a prim carries source identifiers without prior knowledge of a pipeline-specific convention. This argues for a schema-based approach rather than ad-hoc `customData` usage.
6. **External queryability.** Storing a source identifier on a prim is necessary but not sufficient. Real-world workflows also need to resolve the reverse question: *given an external identifier, which USD layers and prims reference it?* The source identifier mechanism should make it tractable for consumers to build external indexes over USD content.
7. **Round-trip fidelity.** Source identifiers should survive a round-trip through USD without loss, even if the characters they contain are not valid in USD prim names.
8. **Minimal disruption.** The solution should build on USD's existing strengths. It should not require fundamental changes to the composition engine or namespace path semantics.

Open questions for discussion

1. Cross-system resolution and indexing.

Once source identifiers are stored in USD, consumers will need to resolve the reverse question: "which layers and prims reference identifier X?" Consumers are responsible for building their own indexes and optimizing for their own access patterns (key-value lookups, dependency graphs, search) on top of whatever identifier mechanism USD provides. What characteristics of the mechanism make that tractable?

2. Dictionary metadata vs. applied schema?

Should source identifiers live as sub-dictionaries within `assetInfo`, or as typed properties on applied schemas that contribute to `UsdPrimDefinition`? (These are labeled Approach A and Approach B respectively in [Likely direction](#).) Key trade-offs:

- **Discoverability and validation:** applied schemas enable GUI presentation of unauthored properties, schema versioning, and schema-driven validation; dictionaries do not.
- **Heterogeneous packages:** Different domains need different identifier fields. A single multi-apply schema must either carry only the common subset or accumulate rarely-used properties -- the more heterogeneous

the contents, the more this tension favors dictionaries or a family of domain-specific schemas.

- **Adoption velocity:** Dictionaries let domains move independently; schemas require agreement on property names before shipping.

See [Likely direction](#) for a detailed comparison.

3. Stratification and governance.

Under either approach, how should vendor and domain extensions be structured and governed? The vendor extension principle implies a tiered lifecycle -- analogous to glTF's vendor → EXT_ → KHR_ promotion path, or OpenGL's GL_NV_ → GL_EXT_ → GL_ARB_ → core -- where vendor-specific conventions can ship immediately, successful patterns are promoted to multi-vendor conventions, and mature conventions become candidates for core standardization. What naming, namespacing, and registration conventions ensure that extensions from different vendors and domains (PLM systems, AECO standards, M&E pipelines, authorship tools) remain discoverable, composable, and non-conflicting -- while allowing vendors to ship without waiting for cross-industry consensus? Concrete governance and registration mechanisms are deferred to the follow-up solution proposal (see [Next steps](#)).

4. Scope: model roots only, or any prim?

The `UsdModelAPI` convenience layer scopes `assetInfo` to model roots, but `assetInfo` itself is registered in `SdfsSchema` for both prims and properties. Migrating the API from `UsdModelAPI` to `UsdObject` (an independently worthwhile change) would eliminate this restriction. Applied schemas naturally apply to any prim already. Either way, external identifiers are needed on prims at all levels of the hierarchy and the mechanism must not be artificially limited to model roots.

5. Namespacing of identifiers.

If a prim carries identifiers from multiple external systems, how should they be organized? The namespacing convention must accommodate the full spectrum of "vendors" -- from a single product (e.g., `windchill`, `revit`) to an industry standard (e.g., `ifc`, `opcua`) to an internal enterprise system. For dictionaries, this means sub-dictionary naming conventions with vendor prefixes. For applied schemas, this means choosing between multi-apply instance names (e.g., `sourceId:revit`, `sourceId:ifc`) or a family of single-apply schemas that include a common base.

6. Relationship to `displayName`.

How does the source identifier relate to the prim's `displayName`? In some workflows the display name *is* the source identifier; in others they differ. When multiple instances share a source identifier, deriving `displayName` from it may be confusing for presentation (e.g., 30 doors all displaying the same catalog number). `displayName` substitution patterns -- as used for symmetry in rigging and for multiple instances of character groups -- may be relevant if this is a needful pattern.

7. Relationship to authorship traceability.

Authorship traceability (tracking who created or modified content) is a related but distinct concern that could build on the same domain identifier pattern. Under either candidate approach, authorship could be one domain-specific use case among many. How should the design accommodate this without conflating authorship metadata with source identification?

8. **Interaction with transcoding.**

If source identifiers contain characters invalid in USD, they should be stored verbatim (not transcoded) in whatever mechanism is chosen. Transcoding is relevant when external identifiers need to be *embedded in prim names*, which is a separate concern. The source identifier mechanism should eliminate the need to encode external identifiers into prim names in most workflows.

Likely direction

TAC discussion has identified two candidate approaches, each with distinct trade-offs. The next phase of this work will evaluate both before committing to one.

Approach A: Extend `assetInfo` with stratified sub-dictionaries.

Domains register source identifiers as sub-dictionaries within `assetInfo`, with applied API schemas providing convenience access (following the `UsdMediaAssetPreviewsAPI` precedent).

Approach B: Applied schema (likely multi-apply) with typed properties.

Applied schemas are a well-established USD mechanism; the question is whether to direct them at the cross-domain identifier problem as a standards framework. Source identifiers would be expressed as properties on an applied API schema, with each external system represented as a schema instance.

	Approach A (<code>assetInfo</code> dictionary)	Approach B (applied schema)
Pros	- Builds on existing composition semantics (element-wise composed, nestable). - Low barrier to adoption -- <code>assetInfo</code> already exists and is familiar. - Freeform dictionaries accommodate heterogeneous identifier packages; domains can move independently and converge on conventions over time.	- Identifier schemes become part of <code>UsdPrimDefinition</code> and <code>UsdTypeInfo</code>, enabling discoverability and GUI presentation of unauthored properties. - Schema versioning and validation are built in. - More rigorous structure may reduce adoption fragmentation across domains.

	Approach A (<code>assetInfo</code> dictionary)	Approach B (applied schema)
Cons	- Convenience API only -- not true data type definitions in <code>UsdPrimDefinition</code>; no fallback values, no automatic GUI presentation, limited validation. - Without active governance, <code>assetInfo</code> becomes a dumping ground of ad-hoc keys. (Both approaches require curation, but the risk is more acute here -- dictionaries impose no structural guardrails.)	- Domains have different identifier fields; a single schema must carry the common subset or accumulate rarely-used properties, slowing adoption. - Requires distributing schema plugins -- though tools already ship their own domain plugins and unrecognized schema data roundtrips without loss. - Without active governance, schema proliferation creates its own discoverability burden.

A possible variant of Approach B is a single-apply base schema in the core (empty or with only common properties), with domain-specific single-apply schemas that include the base and add their own extensions. This encodes systems in concrete schemas rather than multi-apply instance names, but requires `UsdSchemaRegistry` query enhancements.

The remaining open questions above are intended to resolve which approach best serves the community, including scope (any prim vs. model roots), namespacing conventions, and how adjacent use cases (including authorship traceability) should be accommodated.

Risks

- 1. Uncurated proliferation.** Both approaches are vulnerable to unmanaged growth. Under Approach A, ad-hoc keys accumulate in `assetInfo` without structure. Under Approach B, a proliferation of domain-specific schema plugins creates its own discoverability and distribution burden. The mechanism alone does not prevent this -- either approach requires clear governance and curation. But curation itself introduces friction: teams building production pipelines cannot wait for cross-industry consensus before shipping something that works for their domain. The design must find a balance that lets domains move independently while still converging on interoperable conventions over time.
- 2. Premature standardization.** Locking in a pattern before enough domains have exercised it risks discovering too late that it doesn't generalize. The baseline example should be validated against at least two or three distinct domain use cases before being promoted as a standard.
- 3. Adoption fragmentation.** If the mechanism is too generic, domains may still build incompatible conventions on top of it, defeating the interoperability goal. The design must balance flexibility with enough structure to ensure cross-domain discoverability.
- 4. Scope creep.** Source identifiers shade into broader metadata concerns

-- authorship, semantic meaning, lifecycle state -- and the mechanism tries to solve everything. The scope should remain focused on identification and linking; adjacent concerns should build on the pattern as separate extensions, not expand the core.

Relationship to other proposals

This proposal is conceptually upstream of several related efforts:

- [Unicode Identifiers in USD](#)
(Implemented, v24.03) -- Expanded USD's prim name grammar to support Unicode XID characters. This broadens what can be expressed *as a prim name* but does not address the separation of concerns between prim names and external identifiers.
- [Extended Unicode Identifiers](#)
-- Explores further extensions to prim name grammar (leading digits, medial hyphens). This addresses the second problem identified above -- improved ergonomics of path identifiers -- and is a complementary effort. This proposal does not foreclose or deprioritize that work; rather, by establishing a dedicated source identifier mechanism, it reduces the pressure on prim name grammar to accommodate every external naming convention, allowing grammar extensions to be evaluated on their own merits.
- [Bi-Directional Transcoding of Invalid Identifiers](#)
-- Proposes a Bootstring-based algorithm for round-trip encoding of arbitrary UTF-8 strings into valid USD identifiers. Transcoding remains valuable for generating readable prim names from external strings, but a dedicated source identifier mechanism reduces the reliance on transcoding as the *only* way to preserve external identifiers in USD.
- [Revise Use of Layer Metadata](#)
-- Proposes migrating stage metadata to applied schemas. The analysis of `assetInfo`'s prim-based design in that proposal is directly relevant: the same reasoning that led to `assetInfo` being prim-based (persistence through flattening, composability in referenced assets) applies to source identifiers.
- [UI Hints in USD](#)
(Implemented)
-- Has migrated `displayName`, `hidden`, and `displayGroup` into a `uiHints` dictionary, with deprecation of the existing top-level fields. This cements `displayName` as a presentation concern and creates urgency for a dedicated source identifier mechanism, since pipelines currently using `displayName` as a workaround now face active API and content compatibility costs.

Next steps

1. **Submit as pull request.**  Submitted as [PR #105](#).

Community feedback and discussion on the open questions is welcome there.

2. **Align on the problem statement.** Confirm among TAC members and industry stakeholders that the problem is understood consistently and that the framing resonates across industries. The `displayName` deprecation path makes this time-sensitive: every month without a standardized alternative adds to the technical debt accumulating on ad-hoc workarounds.
3. **Gather additional use cases.** Solicit concrete examples from manufacturing, product lifecycle, digital engineering, M&E, and other domains to ensure the framing is not inadvertently biased toward any single industry. Steps 2 and 3 can proceed in parallel.
4. **Evaluate and design the mechanism.** Based on the open questions, evaluate the two candidate approaches -- extending `assetInfo` with stratified domain sub-dictionaries (Approach A) vs. applied schemas with typed properties (Approach B) -- or determine whether a hybrid or alternative mechanism is warranted. Define concrete guidelines and a baseline example in the OpenUSD codebase.
5. **Draft a solution proposal.** Based on alignment from steps 2-4, draft a concrete proposal specifying the mechanism, its composition semantics, and its API -- including the vendor extension governance model (prefix conventions, registration process, promotion criteria) and how adjacent use cases like authorship traceability fit into the pattern.

Stakeholders who want to accelerate this work are encouraged to engage directly on any of the steps above. The pace is determined by the breadth of consensus achieved at each step -- and that consensus is what ensures the solution serves the full community rather than a single use case.

Appendix A: AI-Assisted Drafting

This proposal was drafted with the assistance of an AI language model (Claude, Anthropic) operating within Cursor IDE, under the direction of Aaron Luk. All conceptual framing, editorial decisions, and technical judgment are the responsibility of the human author. The AI was used as a drafting tool to accelerate the writing process based on context and direction provided by the author.

The context provided to the AI was itself the product of extensive preceding discussion among stakeholders and subject matter experts across AOUSD working groups, industry partners, and internal reviewers. The AI did not participate in those discussions; it received their outputs as input for drafting.

Context provided to the AI

The following materials were provided as input context for drafting:

1. **Meeting discussion notes** -- Summary of an AOUSD TAC discussion from 2026-02-20 covering the core problem statement (separating USD identifiers from external identifiers), key questions (instance vs. source, single

value vs. package), existing USD concepts (`assetInfo`), the desired approach (conceptual separation first, implementation details later, cross-industry alignment), and timeline.

2. [Extended Unicode Identifiers proposal](#)

-- Referenced as related but noted as an implementation detail regarding prim name syntax expansion.

3. [Bi-Directional Transcoding of Invalid Identifiers proposal](#)

-- A draft proposal for Bootstring-based encoding of arbitrary UTF-8 strings into valid USD identifiers. Referenced as a related implementation technique.

4. **AOUSD AECO Interest Group Prim Name Grammar Use Case**

(DRAFT v0.0, dated 2025-04-05) -- AECO domain requirements including leading digits, medial hyphens, slashes, GUIDs, regional characters, bi-directional transcoding, and semantically meaningful naming conventions.

5. **Existing proposals in this repository** -- The AI reviewed the structure and conventions of published proposals (e.g., `semantic_schema`, `tf_utf8_identifiers`, `revise_use_of_layer_metadata`) and the `UsdModelAPI` / `assetInfo` documentation to inform the analysis of existing mechanisms.

6. [UI Hints in USD](#) [\(Implemented\)](#)

-- The migration of `displayName`, `hidden`, and `displayGroup` into a `uiHints` dictionary with deprecation of the existing top-level fields. Already implemented in OpenUSD `dev`. Identified as creating urgency due to API and content compatibility implications for pipelines using `displayName` as a workaround for source identifiers.

7. [Composable Bindings: Simplified System Integration](#)

(Microsoft and NVIDIA, November 2025) -- Whitepaper describing an integration pattern for connecting industrial data systems (data lakes, telemetry, OTEL, CloudEvents) to visualization/simulation engines (OpenUSD/Omniverse). Object identifiers from external systems are the binding key to USD scene objects. Illustrates the digital engineering and operational telemetry use case for source identifiers.

8. **Association for Manufacturing Technology (AMT) materials** -- GTC working session proposal "Asset Identity & Geometry-Anchored Semantics for Manufacturing" (AOUSD / AMT / NVIDIA), accompanying notes on identity layers from AMT collaborators, and Feature ID Flowchart (Matt McCormick, AMT). Describes identity continuity as the requirement that source identifiers survive manufacturing state transitions (destructive geometry changes, re-parenting, assembly changes) and extends to feature-level identifiers that may appear, disappear, or deform across process steps. Explicitly aligns with "ongoing AOUSD discussions on decoupling USD identifiers from external system identifiers."

9. **AOUSD IEDT Interest Group presentation (PTC)** -- Steve Ghee (PTC)

presented CAD/PLM integration with USD: direct USD export from Creo and Onshape, automatic USD generation from Windchill PLM on CAD check-in, configuration-driven on-demand USD generation, bidirectional traceability via asset resolvers, and access control flowing through the resolver. Demonstrated with a ~6,000-part tractor from Creo rendered in Omniverse.

10. **PLM identifier architecture (PTC/Windchill)** -- Discussion with Steve Ghee on how PLM identifiers work in practice: Windchill OIDs are intentionally opaque URIs (e.g., `VR:wt.part.WTPart:23639563`); the object identifier, display name, assembly occurrence ID, and system-unique "number" are all distinct concepts; multiple OIDs per item (PLM part, CAD document, navigation context); the asset resolver URI is the real external reference. Part numbers don't scale in complex parametric/configuration use cases -- the real identifier is a composite URI meaningful only to the originating system. Motivated the addition of vendor extensibility as a design principle.

Review and refinement

The draft was refined through multiple rounds of internal review. Key editorial decisions included:

- Explicitly naming the two distinct problems (unencumbered source identifier field vs. improved prim name ergonomics) based on reviewer feedback, and positioning this proposal as addressing the first without foreclosing the second.
- Compressing and reframing the GUIDs discussion to serve the proposal's argument (GUIDs-as-primary-identifiers pressure is a symptom of the missing separation of concerns) rather than reading as a defensive digression.
- Augmenting `assetInfo` analysis with `UsdModelAPI` and `UsdMediaAssetPreviewsAPI` as potential prototypes for a source identifier mechanism.
- Correcting the IFC GlobalId characterization (GlobalIds are per-instance, not per-type) based on fact-checking against the IFC specification.
- Adding external queryability as a design principle and cross-system resolution as an open question, based on reviewer feedback that the practical utility of source identifiers depends on the ability to resolve them from outside USD -- with the consumer responsible for building indexes on top of the mechanism.
- Incorporating TAC feedback: adding authorship traceability as a related use case, updating next steps to reflect consensus trajectory, and adding PR submission as the first next step.
- Incorporating TAC feedback on applied schema limitations: tempering the `UsdMediaAssetPreviewsAPI` pattern's characterization, presenting the case for true applied schemas (multi-apply or single-apply with base) as an alternative to the `assetInfo` dictionary approach, and restructuring the "Likely direction" section to compare both approaches with their trade-offs.

- Adding vendor extensibility as a design principle, referencing cross-industry precedent (Khronos, IETF, W3C) for vendor-first deployment with a path to standardization.
- Adding Khronos glTF as the primary vendor extension precedent (a 3D data interchange format whose extension model governs properties on objects, more directly analogous than the OpenGL/Vulkan graphics API precedent). Explicitly deferring vendor extension governance details (prefix conventions, registration process, promotion criteria) to the follow-up solution proposal.

A prompt-level drafting log has been archived separately.